

# 05 API - Gebruik van Data Transfer Objects (DTO's)

📅 09-09-2026 ⌚ Ongeveer 30 minuten 🗨️ Ongeveer 4500 woorden

---

Tot op heden gebruikte je voor elk request telkens een volledig model om gegevens te tonen of op te vragen, wat overeen komt met complete records vanuit de database. Dit is nochtans niet altijd de bedoeling. Vaak wil je enkel de gegevens die je gebruikt bij requests zo efficiënt mogelijk te maken. bv bij een POST-request moet er geen id of een aanmaakdatum meegegeven worden of je vraagt een lijst op van bestellingen met orderlijnen maar wil de klantId en productId niet zien.

Om dit te verwezelijken, kan er gebruik gemaakt worden van een DTO (Data Transfer Object) , wat een aangepaste klasse is waarin men enkel de benodigde gegevens definiëert. Hierdoor moet niet het hele model gebruikt worden of kan er zelfs data uit meerdere modellen gebruikt worden, waardoor een eenvoudiger en efficiënter gegevensobject ontstaat. Gegevens die niet relevant zijn of niet bedoeld zijn voor client-side applicaties worden dan uit de requests gefilterd.

💡 Je bent ongetwijfeld al het scenario tegengekomen waar je een Model gebruikte om een Create/Insert-statement uit te voeren. Een Insert-statement heeft geen Id nodig (wordt automatisch gegenereerd) waardoor je dit in Swagger steeds moest verwijderen. Of een eindeloze lus exception omdat 2 modellen naar elkaar verwijzen in een GET actie waardoor je JsonIgnore moet toepassen. Dit zijn enkele van de problemen die DTO's trachten op te lossen door objecten aan te bieden die enkel de noodzakelijke Properties voorzien.

Om het verschil tussen DTO's en modellen beter te begrijpen moet er even stapje achteruit gezet worden om al de verschillende varianten van modellen eens onder de loep te nemen.

[Clone de startbestanden indien je deze nog niet hebt. \(Einde hoofdstuk 4\).](#)

## Models, ViewModels, Dto's, Entities, Oh my.

---

Tot nu toe heb je in een applicatie voornamelijk gewerkt met **Models**: eenvoudige C# klassen met Properties en soms methoden om die data te manipuleren. Over het algemeen bevatten Models de data binnen een applicatie. Daarom wordt in het MVC-patroon ( Mo-

del-View-Controller ) het Model beschouwd als het data-gedeelte. In grotere applicaties worden naast modellen ook vaak **DTO's** (Data Transfer Objects), **ViewModels**, en **Entities** gebruikt.

👉 Het is niet ongewoon dat er voor één enkel concept of object meerdere varianten van klassen bestaan. Zo kan een concept zoals een "Bestelling" in een API-applicatie voorkomen als bv. `BestellingEntity`, `BestellingModel`, `BestellingDto`, `AddBestellingDto`, `UpdateBestellingDto`, en `RemoveBestellingDto`. .

Gelukkig zijn deze klassen meestal vrij eenvoudig. Bijna altijd zijn het gewoon C# klassen met **Properties** en soms methoden. (behalve in het MVVM-patroon, waar ViewModels vaak meerdere methoden kunnen bevatten). Bovendien lijken de verschillende varianten, zoals een `Bestelling` en een `BestellingDTO`, vaak sterk op elkaar en delen ze meestal dezelfde **Properties**, of een subset daarvan.

Maar wat zijn nu precies de verschillen tussen deze klassen, en waarom gebruikt men ze überhaupt? Verhoogt dit niet het aantal klassen aanzienlijk? Het antwoord is ja, dit zorgt inderdaad voor meer klassen en dus meer bestanden in de solution. Het nadeel hiervan is dat de codebase groter is en deze misschien iets complexer oogt. Maar het grote voordeel is dat er hiermee een striktere *Separation of Concerns* (SOC) bereikt wordt, wat uiteindelijk leidt tot beter onderhoudbare, uitbreidbare en leesbare code. Zolang je je code netjes gescheiden houdt d.m.v. folders of projecten, hoeft veel code niet onoverzichtelijke code te zijn.

## Model

Modellen zijn code-representaties van real-world objecten of concepten, zoals bestellingen, klanten, producten... Dit zijn klassen met **Properties** die data bevatten, en soms **methoden** om die data te manipuleren.

Een belangrijk kenmerk van modellen is de vrijheid die ze bieden. Je kunt zoveel **Properties** toevoegen als je wilt, zolang ze maar voldoen aan de verwachtingen van de applicatie. Hierdoor zijn modellen vaak simpele klassen zonder extra complexiteit, zoals annotaties of businesslogica. Ze dienen voornamelijk als gegevensdragers die door de businesslogica worden gebruikt.

Ze kunnen methodes bevatten, maar meestal zijn dit eenvoudige methodes die enkel met hun data te maken hebben, zoals b.v. het berekenen van leeftijd op basis van een geboortedatum, of het omzetten van valuta van euro naar dollar.

```
1 // Een model in C#
2 public class Klant
3 {
4     public int Id { get; set; }
5
6     public string Naam { get; set; } = default!;
7
8     public string Voornaam { get; set; } = default!;
9
10    public DateTime AangemaaktDatum { get; set; }
11
12    public List<Bestelling>? Bestellingen { get; set; } = default!;
13 }
```

## Entiteit - ter info

Een entiteit lijkt op een model, maar is een directe representatie van een **database-tabel**. Stel dat je een database-tabel hebt met de naam Klant, en kolommen zoals Id, Naam, Leef-tijd en Afdeling. Dan zou er een C#-klasse moeten zijn die exact overeenkomt met deze structuur.

Entiteiten zijn veel minder flexibel dan modellen. Je kunt niet zomaar Properties toevoegen of verwijderen, omdat Entity Framework verwacht dat de tabel en entiteit een één-op-één-relatie hebben. Daarnaast bevatten entiteiten vaak **extra database-specifieke velden**, zoals **CreatedOn**, **LastUpdated** of **UpdatedBy**, die vaak niet relevant zijn voor de rest van de applicatie, enkel voor de Database. Ook bevatten ze vaak annotaties zoals **Required** en **MaxLength**, die **alleen relevant zijn voor de database**. Dit kan de klasse 'slordig' doen lijken.

Vanwege deze beperkingen wordt er vaak een los model boven op een entiteit gemaakt dat geen van deze restricties heeft, zodat je meer vrijheid hebt.

```
1 // Een Entity in C#
2 public class KlantEntity
3 {
4     [Key] // Geeft aan dat dit de primaire sleutel is.
5           // Elke db row moet een unieke identifier hebben
6     public int Id { get; set; }
7
8     [Required] // Not Null
9     [MaxLength(100)] // Nvarchar(100)
```

```
10     public string Naam { get; set; } = default!;  
11  
12     [MaxLength(150)] // Nvarchar(150), Nullable  
13     public string Voornaam { get; set; } = default!;  
14  
15     public DateTime AangemaaktDatum { get; set; }  
16  
17     public List<BestellingEntity>? Bestellingen { get; set; } = default!;  
18  
19     // Database specifieke kolommen. Niet relevant voor applicatie.  
20     public DateTime CreatedOn { get; set; }  
21  
22     public DateTime ModifiedOn { get; set; }  
23  
24     public string ModifiedBy { get; set; }  
25 }
```

Normaal gesproken worden er enkel Entiteiten in de Repository classes gebruikt om te communiceren van en naar de Database. Ze worden ook gebruikt om de database tabellen op te stellen gedurende de code-first approach met `DbSets<Entity>`. Vandaar ook de naam `Entity Framework`.

👉 Nota: Om de lessen relatief eenvoudig te houden, zullen er **geen Entiteiten gebruikt worden in deze cursus en examens** en wordt er enkel gefocused op de verschillen tussen Modellen en Dto's. Indien er databasecommunicatie via Repositories nodig is zal dit met een Model uitgevoerd worden.

Probeer om in grotere, professionele applicaties te werken met Entiteiten. Op deze manier kan je database- en functionele logica strict gescheiden houden.

## DTO's

In een API-omgeving wordt meestal een extra laag bovenop de modellen toegevoegd in de vorm van DTO's (Data Transfer Objects). Zoals de naam al doet vermoeden, zijn deze bedoeld om data van punt A naar punt B te verplaatsen. DTO's worden gebruikt om gegevens naar de gebruiker te sturen (bijvoorbeeld bij een GET-request), of om gegevens te ontvangen van de gebruiker (bijvoorbeeld bij een POST-request).

Belangrijk is dat DTO's **enkel de benodigde data bevatten**. Als een model Auto bijvoorbeeld een model Merk bevat, zal de DTO voor Auto alleen de relevante eigenschappen bevatten, zoals de Naam van het merk. Overige details zoals het oprichtingsjaar van het merk

of andere auto's van dat merk zijn niet relevant en worden dus niet opgenomen in de DTO.

Voor update-operaties kan een UpdateDTO worden gemaakt die alleen de eigenschappen bevat die daadwerkelijk bijgewerkt kunnen worden. Hetzelfde met het toevoegen van een nieuw object. De Id wordt meestal automatisch gegenereerd dus hoeft een CreateDTO deze niet te bevatten.

- Open de oplossing van les 4 van het WebAPIDemo-project
- Maak in de root van het project een map DTO
- Voeg onderstaande DTO's toe:

In de `KlantBestellingenDto` voeg je Properties toe die de informatie van een klant bevatten en een lijst van `BestellingDTO`'s. Op deze manier kun je de klant en al hun bestellingen tonen aan de gebruiker. Ook hier voeg je enkel de absoluut noodzakelijke informatie toe. De Id-eigenschap laat je dus achterwege. Je kan ook informatie samenvoegen door 'Voor-naam' en 'Naam' samen te voegen tot 'KlantNaam'.

```
1 public class KlantBestellingenDto
2 {
3     public string KlantNaam { get; set; }
4
5     public List<BestellingDTO> Bestellingen { get; set; } = new();
6 }
```

csharp

In de `BestellingDto` voeg je properties toe die relevant zijn voor een bestelling. In dit geval is dat enkel een lijst van producten.

```
1 public class BestellingDto
2 {
3     public List<ProductDto> Producten { get; set; }
4 }
```

csharp

In de `ProductDto` voeg je dan de properties toe die relevant zijn voor een product. Maak dit zo eenvoudig en 'plat' mogelijk. Gebruik zoveel mogelijk primitieve types zoals `string` en `int`, en vermijd verwijzingen naar andere klassen. Indien er toch verwijzingen moeten zijn, beperk dit dan tot andere DTO's.

```
1 public class ProductDto
2 {
3     public double Aantal { get; set; }
4
5     public string ProductNaam { get; set; }
6
7     public string Beschrijving { get; set; }
8
9     public decimal Prijs { get; set; }
10
11     public double Totaal { get; set; }
12 }
```

csharp

⚠ **Examentip:** Het is erg belangrijk om geen Modellen in DTO's te steken. Een DTO in een andere DTO stoppen kan nog net (hoewel sterk afgeraden), maar een Model hoort absoluut niet thuis in een DTO!

## ViewModel

ViewModellen komen pas later aan bod in het MVC gedeelte. Je kan ze wat vergelijken met de hierboven beschreven DTO's met uitzondering dat een ViewModel ook methodes kan aanbieden, in tegenstelling tot DTO's die enkel Properties bevatten, om een bepaalde view te doen laten werken. Een View is een pagina waarmee een gebruiker kan interacteren, zoals een webpagina of tab op een mobiele app.

Kort gezegd bevat een ViewModel alle data en functionaliteit die 1 bepaalde View nodig heeft om te werken.

- Doorgaans bevat een API Entiteiten, Modellen en Dto's.
- Doorgaans bevat een MVC applicatie Entiteiten, Modellen en ViewModellen.

🧠 In een MVVM applicatie ( Model - View - ViewModel ),vooral mobiele applicaties, bestaan Controllers niet. De ViewModellen nemen daar de verantwoordelijkheden van Controllers over.

## DTO's met Data validatie en ModelState

Bij het gebruik van `POST` en `PUT` -DTO's (zoals voor `Create` en `Update` operaties) worden vaak `Data Annotations` toegepast voor eenvoudige datavalidatie. Hiermee kun je bijvoorbeeld minimum- en maximumlengtes van invoer afdwingen, geldige datumbereiken, getallen binnen een bepaalde minimum-en maximum waarde, niet-lege velden, en zelfs reguliere expressies ( `regex` ). Bovendien kun je eigen foutmeldingen specificeren.

Op deze manier kan je de banale validaties uitvoeren met een minimum aan code. Voor complexe validaties zoals nagaan of een product op een tweedehandssite legaal is, of controleren dat een gebruiker een goede reputatie heeft vooraleer deze een advertentie mag plaatsen, moet je nog steeds zelf code schrijven, idealiter in de `Controllers` of in `Services` .

👉 In les 2 heb je reeds `ModelState Validation` gezien op basis van het model. De validatie die je toen op het Model geplaatst hebt, plaats je vanaf nu op de DTO's. `ModelState validation` doe je minstens altijd op de eerste lijn van interactie met de gebruiker. In dit geval de DTO's. Meer validatie is wenselijk, maar minstens de DTO's moeten validatie hebben.

Voeg volgende DTO's toe aan de map `DTO`

```
1 public class AddProductDto
2 {
3     [Required(ErrorMessage = "De naam moet ingevuld worden.")]
4     [MinLength(3, ErrorMessage = "De naam moet minstens 3 tekens
5 bevatten.")]
6     [StringLength(100, ErrorMessage = "De naam mag niet meer dan 100
7 tekens bevatten.")]
8     public string Naam { get; set; } = default!;
9
10    [StringLength(100, ErrorMessage = "De beschrijving mag niet langer
11 zijn dan 100 tekens.")]
12    public string? Beschrijving { get; set; }
13
14    [Required(ErrorMessage = "De prijs van het product is verplicht.")]
15    [Range(0.01, 10000.00, ErrorMessage = "De prijs moet tussen 0,01 en
16 10.000 liggen.")]
17    public decimal Prijs { get; set; }
```

```

18     }
19
20     public class UpdateProductDto
21     {
22         [Required]
23         public int Id { get; set; } // Hier is de Id wel nodig aangezien je
24         moet weten welk product je moet updaten
25
26         [Required(ErrorMessage = "De naam moet ingevuld worden.")]
27         [MinLength(3, ErrorMessage = "De naam moet minstens 3 tekens
28         bevatten.")]
29         [StringLength(100, ErrorMessage = "De naam mag niet meer dan 100
30         tekens bevatten.")]
31         public string Naam { get; set; } = default!;
32
33         [StringLength(100, ErrorMessage = "De beschrijving mag niet langer
34         zijn dan 100 tekens.")]
35         public string? Beschrijving { get; set; }
36
37         [Required(ErrorMessage = "De prijs van het product is verplicht.")]
38         [Range(0.01, 10000.00, ErrorMessage = "De prijs moet tussen 0,01 en
39         10.000 liggen.")]
40         public decimal Prijs { get; set; }
41     }

```

## Overzicht veelgebruikte DataAnnotations in C#

| Attribuut      | Doel / Uitleg                                             | Voorbeeld                                                              |
|----------------|-----------------------------------------------------------|------------------------------------------------------------------------|
| [Key]          | Markeert property als primaire sleutel (Entity Framework) | [Key] public int Id { get; set; }                                      |
| [Required]     | Verplicht veld (mag niet null of leeg zijn)               | [Required] public string Naam { get; set; }                            |
| [StringLength] | Minimale en maximale lengte instellen                     | [StringLength(50, MinimumLength = 3)] public string Naam { get; set; } |
| [MinLength]    | Minimale lengte voor string of collectie                  | [MinLength(8)] public string Wachtwoord { get; set; }                  |



| Attribuut           | Doel / Uitleg                                          | Voorbeeld                                                                                     |
|---------------------|--------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| [MaxLength]         | Maximale lengte voor string of collectie               | <code>[MaxLength(100)] public string Omschrijving { get; set; }</code>                        |
| [Range]             | Waarde binnen een bereik (ook voor DateTime)           | <code>[Range(1, 100)] public int Aantal { get; set; }</code>                                  |
| [RegularExpression] | Controle op patroon (regex)                            | <code>[RegularExpression(@"^[0-9]{4}[A-Z]{2}\$")] public string Postcode { get; set; }</code> |
| [EmailAddress]      | Controle op geldig e-mailadres                         | <code>[EmailAddress] public string Email { get; set; }</code>                                 |
| [Phone]             | Controle op geldig telefoonnummer                      | <code>[Phone] public string Telefoonnummer { get; set; }</code>                               |
| [Url]               | Controle op geldige URL                                | <code>[Url] public string Homepage { get; set; }</code>                                       |
| [Compare]           | Vergelijkt waarde met een andere property              | <code>[Compare("Wachtwoord")] public string BevestigWachtwoord { get; set; }</code>           |
| [Display]           | Geeft een alternatieve naam of label voor een property | <code>[Display(Name = "Volledige naam")] public string Naam { get; set; }</code>              |

💡 Bij GET-DTO's zijn dergelijke validaties overbodig (behalve dan [ Display ]) omdat je enkel data presenteert aan de gebruiker en geen input valideert.

In de controllers kun je met `ModelState.IsValid` controleren of een ingevoerde DTO geldig is. Als de validatie faalt, kun je met een `BadRequest`-response de fouten aan de gebruiker teruggeven.

- Vervang in de `ProductController` de parameters van de `AddProduct`-methode door `AddProductDto ProductDto` .

```
1 // POST: api/Product csharp
2 [HttpPost]
3 public IActionResult AddProduct(AddProductDto productDto)
4 {
5     // Controleer of het model voldoet aan de validatie
6     if (!ModelState.IsValid)
7     {
8         // Geeft een 400 Bad Request terug met de modelvalidatiefouten
9         return BadRequest(ModelState);
10    }
11 }
```

Hoewel Data annotaties snel en eenvoudig fouten kunnen opsporen en weergeven is het belangrijk om te onthouden dat deze alleen geschikt zijn om de vorm en inhoud van gebruikersinput te valideren. Ze bieden een eenvoudige manier om gewone validatie uit te voeren, zoals het controleren van vereiste velden, minimale en maximale lengte van strings, getalbereiken, enzovoort. Het biedt nauwelijks bescherming tegen gerichte aanvallen.

Kritieke validaties die verder gaan dan inputcontrole moeten nog steeds worden uitgevoerd in de `business logica`, in dit geval de `Controllers`:

- **Authorisatie:** Controleren of een gebruiker de juiste rechten heeft om een bepaalde actie uit te voeren (bijv. alleen beheerders mogen gegevens verwijderen).
- **Business regels:** Specifieke business logica, zoals ervoor zorgen dat een gebruiker niet meer geld opneemt dan op zijn rekening staat, of dat een product niet wordt verkocht als het uit voorraad is.

Hier is het systeem namelijk in staat om niet alleen naar de input zelf te kijken, maar ook naar de bredere context, zoals gebruikersrechten, databasestatusen en bedrijfsregels.

Nota: In een 3-layered systeem of vergelijkbare architectuur worden Data Annotations vaak toegepast op DTO's voor inputvalidatie en op Entities om Entity Framework te instrueren hoe de databasekolommen eruit moeten zien. Het is echter goed om de modellen zo eenvoudig mogelijk te houden door geen annotations in de modellen zelf te plaatsen. Op deze manier blijven ze eenvoudig en overzichtelijk.

👉 Omdat deze cursus geen gebruik maakt van entiteiten, worden de annotations direct op de DTO's en modellen geplaatst. Dit is echter geen best practice en werd lauter toegepast om de hoeveelheid code en complexiteit te beperken.

# Mapping

Nu je verschillende varianten van hetzelfde concept, zoals "Product", gebruikt, moet je vaak data 'verplaatsen' tussen deze varianten. Dit proces gebeurt meestal als volgt: Entity -> Model -> DTO voor **GET**-operaties of DTO -> Model -> Entity voor **POST**-operaties.

Om dit te bereiken, moet je de data van het ene object naar het andere kopiëren. Dit proces staat bekend als Mapping. In de praktijk komt dit erop neer dat elke Property van object A handmatig toegewezen moet worden aan de overeenkomstige Property in object B. Dit herhaal je voor alle Properties, totdat de volledige data is overgezet.

```
1 // POST: api/Product csharp
2 [HttpPost]
3 public IActionResult AddProduct(AddProductDto productDto)
4 {
5     // ModelState-validatie
6
7     Product product = new Product
8     {
9         Naam = dto.Naam,
10        Beschrijving = dto.Beschrijving,
11        Prijs = dto.Prijs
12        ...
13    };
14
15    await _unitOfWork.ProductRepository.AddAsync(product);
16    ...
17 }
```

👉 Zoals je kunt zien, is dit een relatief arbeidsintensief proces. Het is niet alleen foutgevoelig, maar ook tijdrovend en moeilijk te onderhouden. Voor elke nieuwe Property die toegevoegd of verwijderd wordt, moet de Mapping handmatig worden bijgewerkt. Dit leidt vaak tot veel duplicatie van code, of Properties die vergeten gemapt te worden, wat de leesbaarheid en betrouwbaarheid vermindert, om nog niet te spreken van de eindeloze 'saaie' lijnen code die quasi allemaal hetzelfde doen.

# AutoMapper

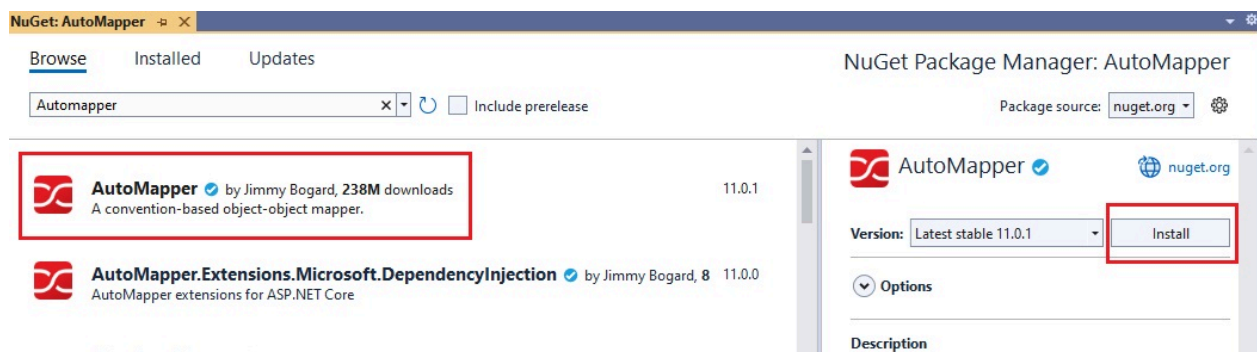
Een veel betere aanpak is om deze "*plumbing code*" te automatiseren met een tool zoals AutoMapper . Met AutoMapper kan je eenvoudig en automatisch Properties van het ene object naar het andere kopiëren, zolang de Properties **dezelfde naam en hetzelfde type hebben**. Voor specifieke gevallen waar manuele aanpassingen nodig zijn, biedt AutoMapper ook de mogelijkheid om de Mapping handmatig te configureren. Dit maakt de code veel korter, overzichtelijker en minder foutgevoelig.

## Installatie

Allereerst moet je de AutoMapper package installeren

### Visual Studio

*Tools -> Nuget Package Manager.*



### Terminal

1

```
dotnet add package AutoMapper
```

Nadien moet je een nieuwe Profile klasse opstellen.

- Voeg een nieuwe map Configuration toe.

- Voeg hierin een nieuwe klasse `MapperProfile` toe en laat ze overerven van `AutoMapper.Profile`.

In de constructor kan je met de `CreateMap<Source, Destination>` methoden instellen welke klasse mapbaar is naar een andere. Als je de code op deze manier instelt, kan je bv een `JobEntiteit` mappen naar een `JobModel`. Als je ook het model mapbaar wilt maken naar de `Entiteit` (wat vaak gebeurt) kan je `ReverseMap()` toevoegen aan de lijn.

```

1  using AutoMapper;
2
3  public class MappingProfile : Profile
4  {
5      public MappingProfile()
6      {
7          // Definieer hier je mappings
8          CreateMap<AddProductDto, Product>(); // Deze klassen hebben
9          dezelfde properties. Expliciete mapping is hier niet nodig.
10     }
11 }
```

De volgende stap is AutoMapper injecteren in de applicatie.

- Ga naar `Program.cs` en voeg de volgende lijn toe, voor `var app = builder.Build();`

```

1  builder.Services.AddAutoMapper(AppDomain.CurrentDomain.GetAssemblies());
```

Nu kan er een `IMapper` interface geïnjecteerd worden in eender welke Controller die de functionaliteit nodig heeft via `Dependency Injection`. Op deze manier kunnen de lijnen mapping code aanzienlijk ingekort worden.

- Injecteer de `IMapper` in de `ProductController`

```

1  public class ProductController : ControllerBase
2  {
3      private readonly IUnitOfWork _unitOfWork;
4      private readonly IMapper _mapper;
```

```
5     public ProductController(IUnitOfWork unitOfWork, IMapper mapper)
6     {
7         _unitOfWork = unitOfWork;
8         _mapper = mapper;
9     }
10 }
11
```

- Pas de methode `ProductToevoegen` aan zodat je de AutoMapper gebruikt om `AddProductDto` te mappen naar `Product`

```
1 [HttpPost] csharp
2 public async Task<ActionResult<Product>> ProductToevoegen(AddProductDto
3 productDto)
4 {
5     // ModelState - validatie
6
7     Product product = _mapper.Map<Product>(productDto);
8
9     await _unitOfWork.ProductRepository.AddAsync(product);
10     ...
11 }
```

Soms kunnen niet alle properties automatisch worden gemapt door AutoMapper, bijvoorbeeld als de propertynamen of types niet overeenkomen. In dergelijke gevallen kun je gebruik maken van de `ForMember()` -methode om handmatig te specificeren hoe een bepaalde property gemapt moet worden. Dit lijkt misschien ingewikkeld, maar het komt eigenlijk neer op het volgende patroon: `.ForMember(destination, MapFrom(source))`. De `ForMember` methode kan tot in het oneindige toegevoegd worden aan een enkele `CreateMap`.

```
1 CreateMap<Destination, Source>() csharp
2     .ForMember(dest => dest.DestinationProperty, opt => opt.MapFrom(src
3     => src.SourceProperty));
```

## Gegevens ophalen uit meerdere entiteiten

In het volgende voorbeeld worden de voornaam en naam van de klant gemapt naar de volledige KlantNaam. De mapping voor bestellingen is iets complexer, omdat de property-namen niet overeenkomen of verborgen zitten in gerelateerde klassen. Hiervoor moet handmatige configuratie worden toegevoegd om AutoMapper te vertellen waar de nodige data zich bevindt.

```
1 namespace WebAPIDemo.Configuration csharp
2 {
3     public class MapperProfile : Profile
4     {
5         public MapperProfile()
6         {
7             CreateMap<AddProductDto, Product>(); // Deze klassen hebben
8             dezelfde properties. Expliciete mapping is hier niet nodig.
9
10            // Deze klassen hebben niet dezelfde properties, dus je moet
11            expliciet mappen.
12            CreateMap<Klant, KlantBestellingDto>()
13                .ForMember(dest => dest.KlantNaam, opt => opt.MapFrom(src
14                => $"{src.Voornaam} {src.Naam}"));
15
16            CreateMap<Bestelling, BestellingDto>()
17                .ForMember(dest => dest.Producten, opt => opt.MapFrom(src
18                => src.Orderlijnen.Select(ol => new ProductDto
19                {
20                    Aantal = ol.Aantal,
21                    ProductNaam = ol.Product.Naam,
22                    Prijs = ol.Product.Prijs,
23                    Totaal = ol.Aantal *
24                    Convert.ToDouble(ol.Product.Prijs),
25                    Beschrijving = ol.Product.Beschrijving
26                }).ToList()));
27        }
28    }
29 }
```

### Info

- 🚨 Probeer complexiteit in je MapperProfile(s) zoveel mogelijk te beperken aangezien deze moeilijk te debuggen zijn. Complexe berekeningen of samenvoegingen horen hier niet thuis. Dat doe je in Controllers of Services. Vermijd *magic happens here* code.
- 🧠 Ter info: Mapping werkt default enkel van de Source naar Destination richting. Een source kan omgezet worden naar een destination. Omgekeerd niet, tenzij je `ReverseMap()` toevoegt aan het einde van een mappingConfiguratie. Dit wordt veel gebruikt in de configuratie tussen een Model en een Entiteit aangezien de relatie zowel gebruikt kan worden om data op te halen en weg te schrijven. DTO's werken meestal maar in 1 richting: data ophalen of wegschrijven.
- 🙌 Let erop dat je geen circulaire data toevoegt aan je DTO's! (d.w.z. dto A heeft een B Dto, B heeft een A). Dat zorgt bij het omzetten naar JSON voor een eindeloze lus en dus een Exception.
- Dit is een probleem dat vaak voorkomt als je Modellen probeert te sturen naar je gebruiker en is dus één van de problemen die DTO's trachten op te lossen! Long story short: vermijd het gebruik van klassen binnen DTO's zoveel mogelijk.

- Pas nu de KlantController aan: injecteer de Automapper-service en pas de methoden aan.

```
1 namespace WebAPIDemo.Controllers
2 {
3     [Route("api/[controller]")]
4     [ApiController]
5     public class KlantController : ControllerBase
6     {
7         private readonly IUnitOfWork _uow;
8         private readonly IMapper _mapper;
9
10        public KlantController(IUnitOfWork uow, IMapper mapper)
11        {
12            _uow = uow;
13            _mapper = mapper;
14        }
15
16        [HttpGet("KlantMetBestellingen")]
17        public async Task<ActionResult<IEnumerable<KlantBestellingDto>>>
18        GetKlantenMetBestellingen()
19        {
```



```
20         List<Klant> models = await
21         _uow.KlantRepository.GetKlantenMetBestellingen();
22         if (models.Count == 0)
23         {
24             return NotFound();
25         }
26
27         List<KlantBestellingDto> dtos =
28         _mapper.Map<List<KlantBestellingDto>>(models);
29
30         return Ok(dtos);
31     }
32
33     [HttpGet("KlantMetBestellingen{id}")]
34     public async Task<ActionResult<KlantBestellingDto>>
35     GetKlantMetBestellingen(int id)
36     {
37         Klant? model = await
38         _uow.KlantRepository.GetKlantMetBestellingen(id);
39
40         if (model == null)
41         {
42             return NotFound();
43         }
44
45         KlantBestellingDto dto = _mapper.Map<KlantBestellingDto>
46         (model);
47
48         return Ok(dto);
49     }
50
51     // ...
52 }
```

Merk op dat dezelfde aanpak kan worden gebruikt voor het ophalen van meerdere klanten met hun bestellingen. Hoewel er geen mapping is gedefinieerd voor een lijst, werkt het mappen van een lijst van DTO's automatisch, omdat AutoMapper in staat is om voor elk individueel object een mapping uit te voeren. Indien een enkel object gemapt kan worden, kan een collectie dat ook, zonder verdere configuratie.

Het ophalen van de Modellen is de verantwoordelijkheid van de Repositories en het UnitOfWork patroon.

- Breidt de interface `KlantRepository` uit met onderstaande methodes om de data op te halen.

```
1  public interface IKlantRepository : IGenericRepository<Klant>      csharp
2  {
3      Task<List<Klant>> GetKlantenMetBestellingen();
4
5      Task<Klant?> GetKlantMetBestellingen(int id);
6  }
7
8  public class KlantRepository : GenericRepository<Klant>, IKlantRepository
9  {
10     public KlantRepository(WebAPIDemoContext context) : base(context) { }
11
12     public async Task<List<Klant>> GetKlantenMetBestellingen()
13     {
14         return await _context.Klanten
15             .Include(x => x.Bestellingen)
16             .ThenInclude(x => x.Orderlijnen)
17             .ThenInclude(x => x.Product)
18             .ToListAsync();
19     }
20
21     public async Task<Klant?> GetKlantMetBestellingen(int id)
22     {
23         return await _context.Klanten
24             .Include(x => x.Bestellingen)
25             .ThenInclude(x => x.Orderlijnen)
26             .ThenInclude(x => x.Product)
27             .FirstOrDefaultAsync(x => x.Id == id);
28     }
29 }
```

🧠 Ter info: Moeite met het begrijpen van bovenstaande `Include`, `ThenInclude` commandos? Probeer het eens naar SQL om te zetten:

```
1 SELECT
2     k.KlantId, k.Naam AS KlantNaam,
3     b.BestellingId, b.Datum,
4     ol.OrderlijnId, ol.Aantal,
5     p.ProductId, p.Naam AS ProductNaam, p.Prijs, p.Beschrijving
6 FROM Klanten k
7 LEFT JOIN Bestellingen b ON k.KlantId = b.KlantId
8 LEFT JOIN Orderlijnen ol ON b.BestellingId = ol.BestellingId
9 LEFT JOIN Producten p ON ol.ProductId = p.ProductId;
```

sql

## Updaten van entiteiten

Om te updaten, moet je zoals je aangeleerd hebt, éérst de inkomende gegevens valideren. Daarna zoek je op of het record bestaat in de database. De volgende stap is het mappen van de nieuwe gegevens over de oude gegevens (regel 16). Daarna probeer je de wijzigingen op te slaan in de database.

Pas volgende code aan in je ProductController.

```
1 [HttpPut("{id}")]
2 public async Task<ActionResult> ProductWijzigen(int id, UpdateProductDto
3 updateProductDto)
4 {
5     if (id != updateProductDto.Id)
6     {
7         return BadRequest("De opgegeven id's komen niet overeen.");
8     }
9
10    if(!ModelState.IsValid)
11        return BadRequest(ModelState);
12
13    Product? product = await
14    _unitOfWork.ProductRepository.GetByIdAsync(id);
15    if (product == null)
16        return BadRequest("Er is geen record gevonden met dit Id.");
17
18    _mapper.Map(updateProductDto, product);
19
20    _unitOfWork.ProductRepository.Update(product);
21
22    try
```

CS

```
23     {
24         _unitOfWork.SaveChanges();
25     }
26     catch (DbUpdateConcurrencyException)
27     {
28         var producten = await
29     _unitOfWork.ProductRepository.GetAllAsync();
30         if (producten.Any(x => x.Id == id))
31         {
32             return NotFound("Er is geen product met dit id gevonden");
33         }
34         else
35         {
36             throw;
37         }
38     }
39
40     return NoContent();
41 }
```

### Info

Merk op dat er een verschil zit tussen ophalen/aanmaken van een object en wijzigen.

`_mapper.Map<Product>(dto)` maakt een nieuw object aan op basis van de DTO; dit gebruik je meestal bij POST / GET. bv. `Product product = _mapper.Map<Product>(productDto);`

`_mapper.Map(dto, existingProduct)` werkt een bestaand object bij met de waarden uit de DTO; dit gebruik je bij PUT. Het verschil zit dus in nieuw maken vs bijwerken van een object.

## Conclusie

Het gebruik van DTO's biedt aanzienlijke voordelen in moderne API's en applicaties. Ze zorgen voor een striktere scheiding van verantwoordelijkheden en maken de communicatie tussen de backend en frontend efficiënter en veiliger. Door alleen de noodzakelijke gegevens naar de client te sturen of te ontvangen, verbeter je de prestaties en beveiliging van jouw applicatie.

Bovendien maakt AutoMapper het overzetten van gegevens tussen verschillende objecten veel eenvoudiger en minder foutgevoelig. Het bespaart ontwikkelaars tijd en moeite, vooral bij applicaties waar veel modellen, entiteiten en DTO's in gebruik zijn. Handmatige

mapping kan arbeidsintensief en foutgevoelig zijn, terwijl AutoMapper het proces automatisch afhandelt en eenvoudig te configureren is.

In combinatie zorgen deze lagen voor een gestructureerde, schaalbare en goed onderhoudbare applicatie, met een duidelijke scheiding tussen database-interactie, businesslogica, en gegevensuitwisseling.

## Kortom

- Entiteiten zijn directe representaties van database-tabellen en bevatten alle eigenschappen die nodig zijn om de structuur van de database te weerspiegelen. Ze bevatten vaak extra informatie zoals timestamps (bijv. CreatedOn, LastUpdated) en annotaties voor validatie of database-instellingen (bijv. Required, MaxLength). Entiteiten zijn essentieel voor het werken met ORM-tools zoals Entity Framework, maar kunnen inflexibel zijn omdat ze de database nauw moeten volgen.
- Modellen bieden meer vrijheid dan entiteiten en worden gebruikt om de businesslogica te vertegenwoordigen. Ze bevatten alleen de data die nodig is voor de toepassing, zonder de beperkingen van de database. Dit maakt modellen flexibeler en beter bruikbaar in applicatielogica die niet direct met de database of client communiceert.
- DTO's worden gebruikt voor communicatie tussen de backend en frontend (Bv. Api & React app) en kunnen makkelijk van en naar JSON of XML geconverteerd worden. Ze bevatten alleen de relevante gegevens voor specifieke API-operaties. Ze helpen om de applicatie efficiënter en veiliger te maken door overbodige of gevoelige gegevens uit te filteren.
- AutoMapper automatiseert het overzetten van gegevens tussen modellen en DTO's. Dit vermindert duplicatie van code en het risico op fouten, en maakt de code veel eenvoudiger en onderhoudbaarder, vooral in grotere projecten.
- AutoMapper kan automatisch Properties overzetten die dezelfde naam en type hebben. Voor een manuele mapping kan je gebruik maken van `.ForMember(x => destProperty, y => mapFrom(z => srcProperty)) ;`
- Data Annotations zijn handig voor basisvalidatie van gebruikersinput, zoals lengte- en formaatcontroles. Deze annotaties fungeren als een eerste verdedigingslinie en worden meestal toegepast op DTO's en entiteiten in een 3-laags systeem.
- Zaken als autorisatie en bedrijfsregels zijn te complex om alleen door Data Annotations behandeld te worden.

[Hier vind je het WebApiDemo-project.](https://csharp.pit-graduates.be/lessen/NET-development/05-api-dto.html)

**Laatst bijgewerkt:**09-09-2026, 08:52

**Bijdragers:** JorenSynaeveTM